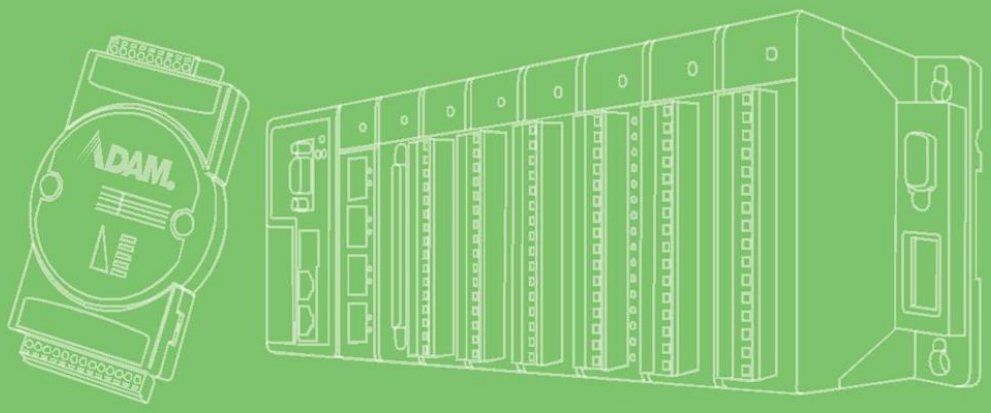


User Manual

Advantech DataConnect Class Library

V1.5

CONTENTS

1. Overview 8

2. Introduction of ADC 9

   2.1. Access the symbols 9

   2.2. Access the flag memory 11

   2.3. Accessing Flag Memory via Symbols 11

   2.4. Data connection and sectors 13

   2.5. Synchronous Communication Mechanism 15

3. ADC common Class 16

   3.1. ADCBasic 16

   3.2. AddDataConnect 16

   3.3. GetVersion 17

   3.4. SetADCPriority 17

   3.5. StartDataPolling 18

   3.6. StopDataPolling 18

   3.7. ShowConnection 19

   3.8. ReadSingleMemoryValue_Local 19

   3.9. ReadMultipleMemoryValue_Local 20

   3.10. ReadMultipleMemoryValue_Local<T> 20

   3.11. WriteSingleMemoryValue_Local 21

   3.12. WriteMultipleMemoryValue_Local 22

   3.13. WriteMultipleMemoryValue_Local<T> 23

4. ADC DataConnect Class 24

4.1. DataConnect 24

4.2. SetMemConnectPort 25

4.3. ConnectSync 25

4.4. DisconnectSync 26

4.5. ConnectionCompleted 26

4.6. DisConnectionCompleted 27

4.7. AddSymdataSector 27

4.8. AddMemdataSector 28

4.9. GetSymbolList 28

4.9.1. ADC_SYMBOL_INFO 29

4.10. ReadSymbolValue 29

4.11. ReadSymbolValue<T> 30

4.12. WriteSymbolValue 31

4.13. WriteSymbolValue<T> 32

4.14. ReadSymbolValueSync 33

4.15. ReadSymbolValueSync<T> 34

4.16. WriteSymbolValueSync 35

4.17. WriteSymbolValueSync<T> 35

4.18. ReadSingleMemoryValue 36

4.19. ReadMultipleMemoryValue 37

4.20. ReadMultipleMemoryValue<T> 38

4.21. WriteSingleMemoryValue 39

4.22. WriteMultipleMemoryValue 40

4.23. WriteMultipleMemoryValue <T> 41

4.24. ReadMemoryValueSync 42

4.25. ReadMemoryValueSync<T> 43

4.26. WriteMemoryValueSync 44

4.27. WriteMemoryValueSync<T> 45

5. ADC Symdata Sector Class 46

5.1. SymdataSector 46

5.2. SymdataReadCompleted 46

5.3. AddSymbol 47

6. ADC Memdata Sector Class 48

6.1. MemdataSector 48

6.2. MemdataReadCompleted 49

7. ADC Conversion Tool Class 50

7.1. ByteArrayConvertToString 50

7.2. ByteArrayConvertToUint16Array 50

7.3. ByteArrayConvertToUint32Array 51

7.4. ByteArrayConvertToUint64Array 51

7.5. ByteArrayConvertToSbyteArray 52

7.6. ByteArrayConvertToInt16Array 52

7.7. ByteArrayConvertToInt32Array 53

7.8. ByteArrayConvertToInt64Array 53

7.9. ByteArrayConvertToFloatArray 54

7.10. ByteArrayConvertToDoubleArray 54

7.11. StringConvertToByteArray 55

7.12. StringConvertToByteArray 55

7.13. Uint16ArrayConvertToByteArray 56

7.14. Uint32ArrayConvertToByteArray 56

7.15. Uint64ArrayConvertToByteArray 57

7.16. SbyteArrayConvertToByteArray 57

7.17. Int16ArrayConvertToByteArray 58

7.18. Int32ArrayConvertToByteArray 58

7.19. Int64ArrayConvertToByteArray 59

7.20. FloatArrayConvertToByteArray 59

7.21. DoubleArrayConvertToByteArray 60

8. ADC delegate functions 61

8.1. OnConnectCompletedEvent 61

8.2. OnDisconnectCompletedEvent 61

8.3. OnSymdataReadCompletedEvent 62

8.4. OnMemdataReadCompletedEvent 63

8.5. OnSymdataWriteCompletedEvent 63

8.6. OnMemdataWriteCompletedEvent 64

9. ADC calling flow 65

9.1. Asynchronous ADC calling flow 65

9.2. Synchronous ADC calling flow 66

10. ADC sample code 67

10.1. Basic example 67

10.2. Symbol example generate 68

10.3. Access local flag memory 70

10.4. Synchronous Access 71

10.5. Synchronous Access Using Generic APIs 71

10.6. STRUCT Access 72

11. ADC library error code 74

Troubleshooting 77

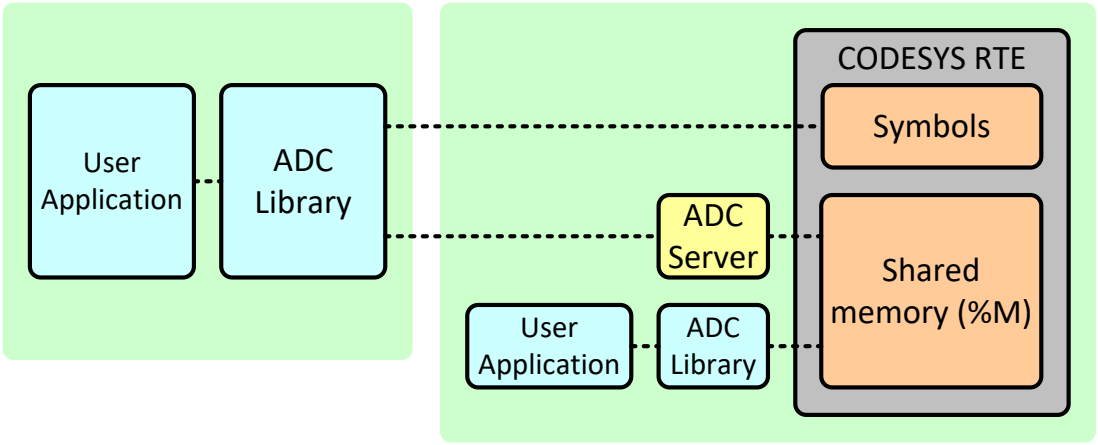
1. PLC channel information 77

Document History:

Doc Version	Date	Comment
1.0	2020/06/24	First edition
1.1	2020/07/20	Add ADC_PLC_NOT_VALID_DEVICE error code
1.2	2020/08/24	1. Add local memory access APIs 2. Added part of convert API
1.3	2022/01/17	1. PLC configuration support username/password login
1.4	2022/8/25	1. Modify ConnectionCompleted, DisConnectionCompleted, SymdataReadCompleted , MemdataReadCompleted event type 2. Modify flag memory total size to 2M bytes
1.5	2026/10/13	Add synchronous and generic APIs, Symbol list, Memory port settings, calling flows, examples and error codes.

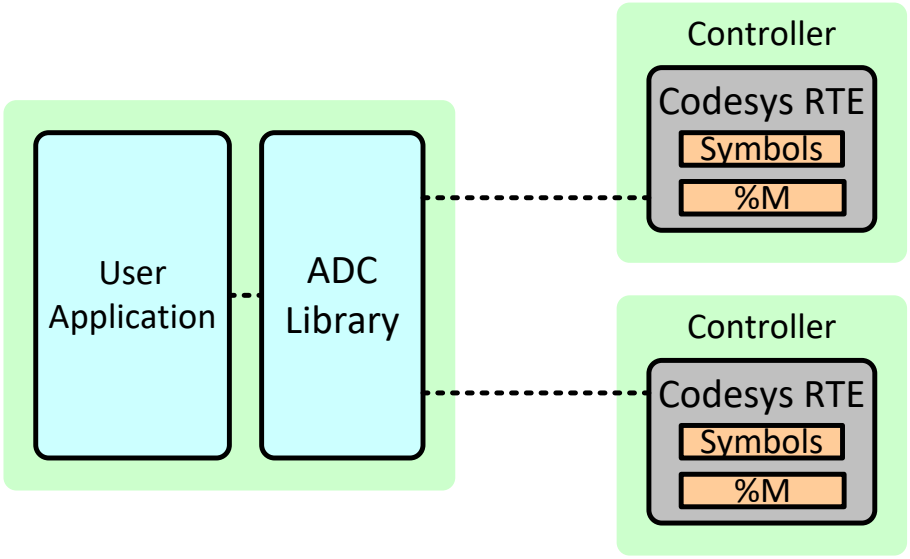
1. Overview

This document describes the Advantech DataConnect Library user manual (Hereinafter referred to as the ADC). ADC is a software interface (API) used for communication between the CODESYS controllers and third-party systems on Windows based system. ADC supports remote access to symbol and flag memory area through the network. And access the local flag memory area at a faster speed (Must be the same device as CODESYS RTE).



ADC also supports that connect to multiple controller

ADC supports asynchronous polling and synchronous communication for PLC Symbols and Flag Memory.

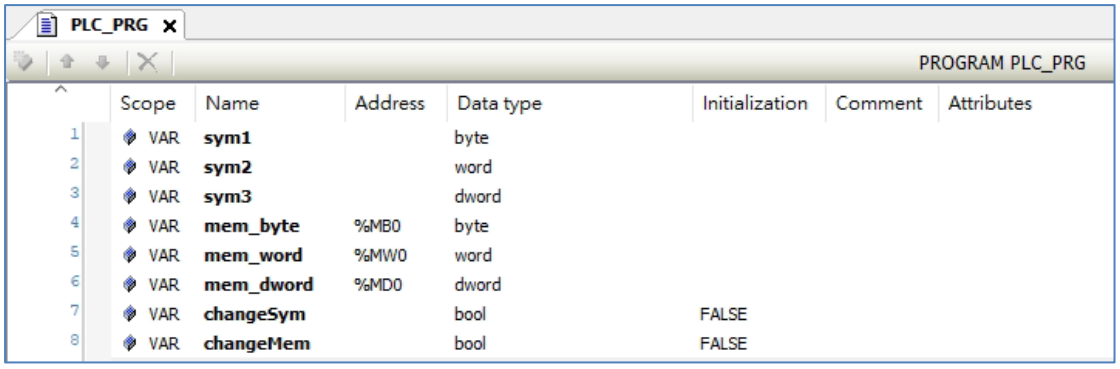


2. Introduction of ADC

Before starting to use ADC library, user need to create CODESYS symbols or define flag memory for variables. And understand the relationship between the ADC connections and sectors.

2.1. Access the symbols

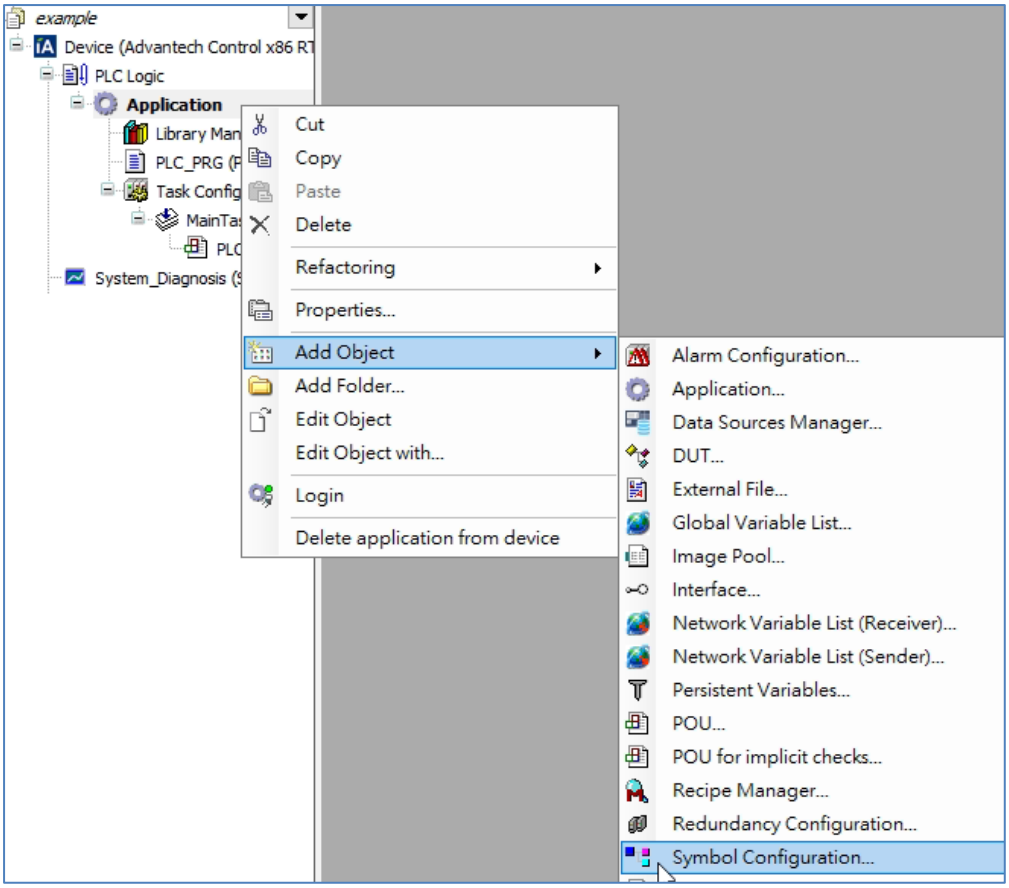
Step1: Create variables



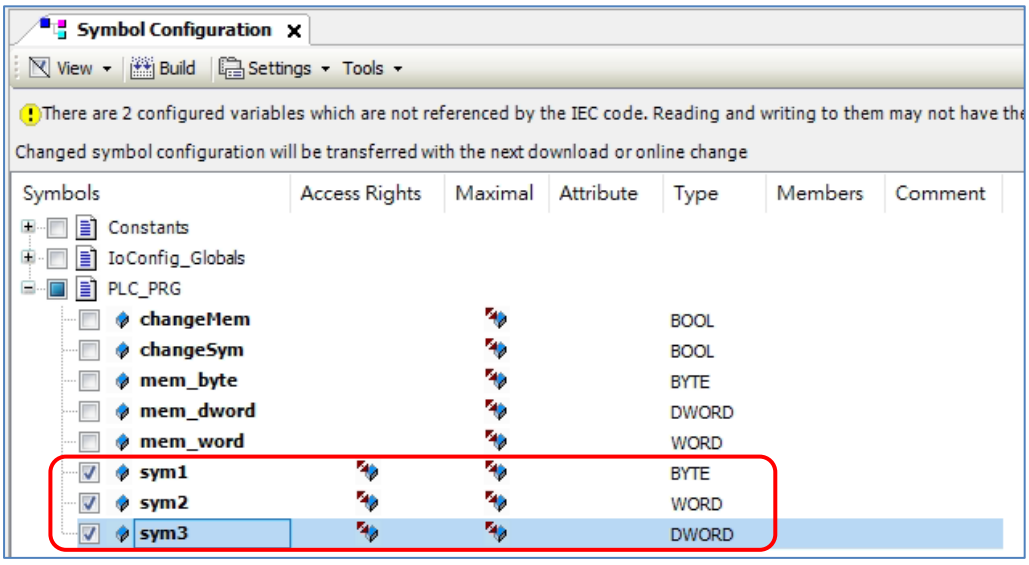
	Scope	Name	Address	Data type	Initialization	Comment	Attributes
1	VAR	**sym1**		byte			
2	VAR	**sym2**		word			
3	VAR	**sym3**		dword			
4	VAR	**mem_byte**	%MB0	byte			
5	VAR	**mem_word**	%MW0	word			
6	VAR	**mem_dword**	%MD0	dword			
7	VAR	**changeSym**		bool	FALSE		
8	VAR	**changeMem**		bool	FALSE		

Step2: Add Symbol Configuration

Note: The total number of symbols for each data connection is 1000.



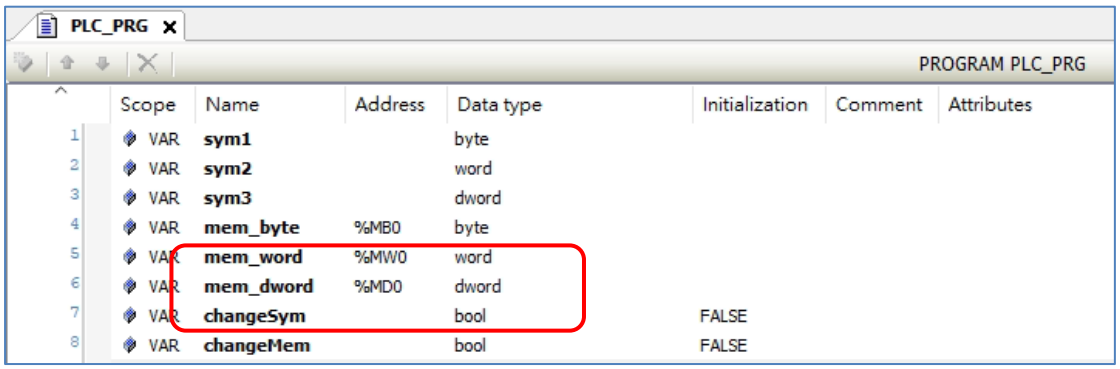
Step3: Select symbols



Step4: Login and download project

2.2. Access the flag memory

Step1: Create variables and map to %M address

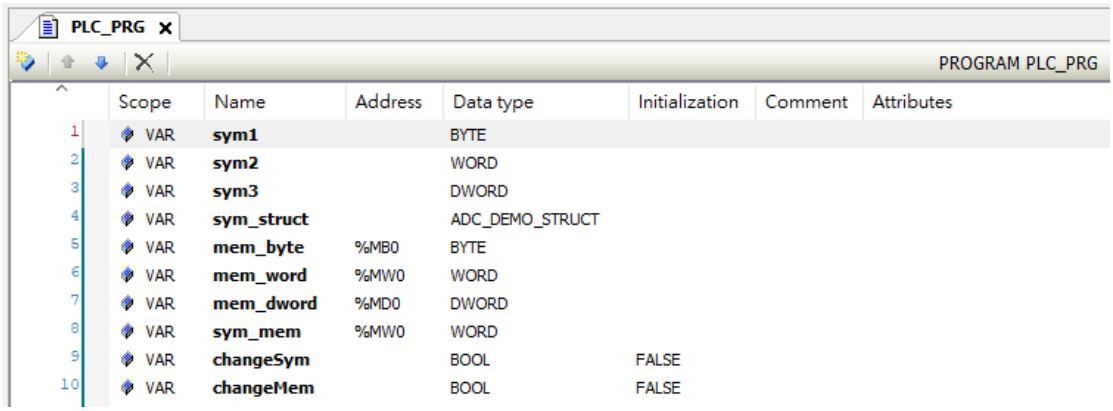


Step2: Login and download project

Note: The total size of the flag memory is 2097152 bytes.

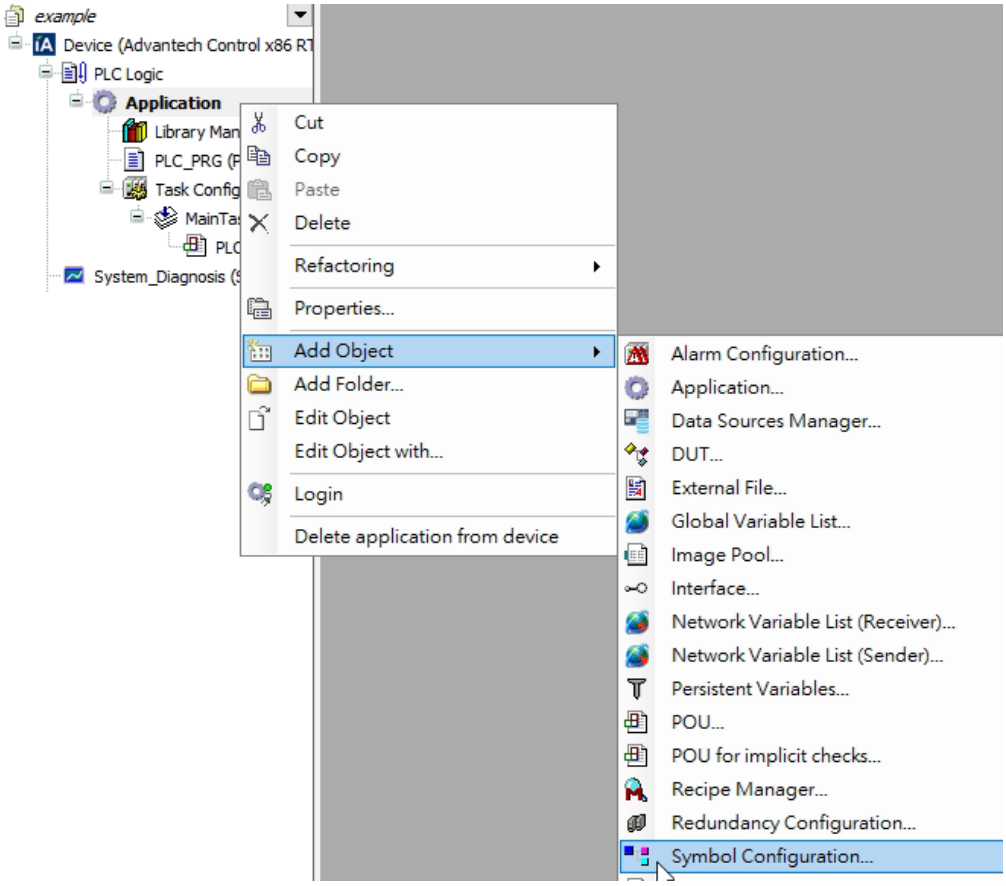
2.3. Accessing Flag Memory via Symbols

Step1: Create variables and map to %M address

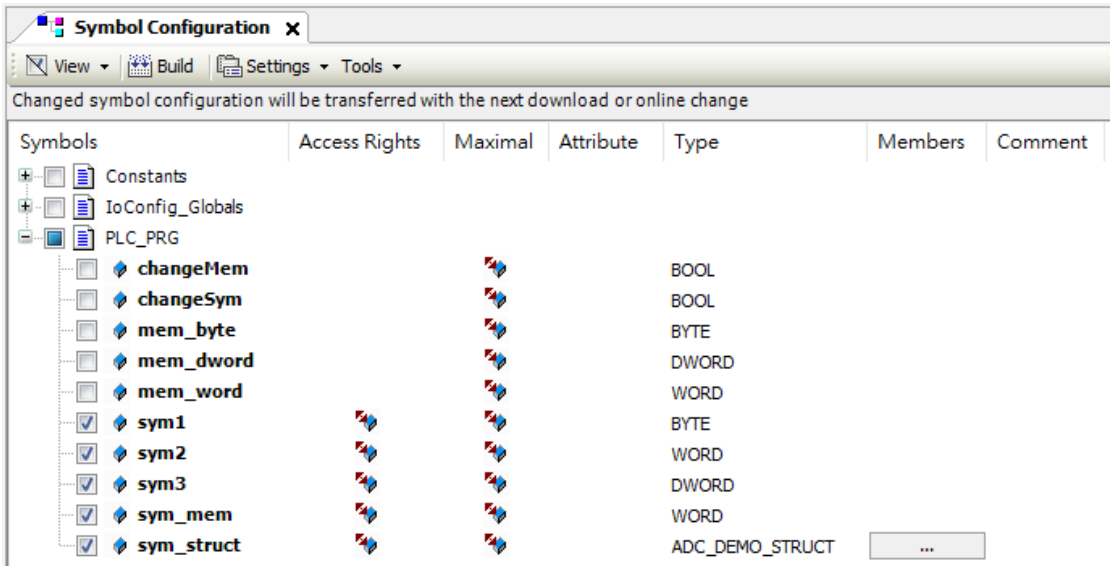


Step2: Add Symbol Configuration

Note: The total number of symbols for each data connection is 1000.



Step3: Select symbols and Build



Step4: Login and download project

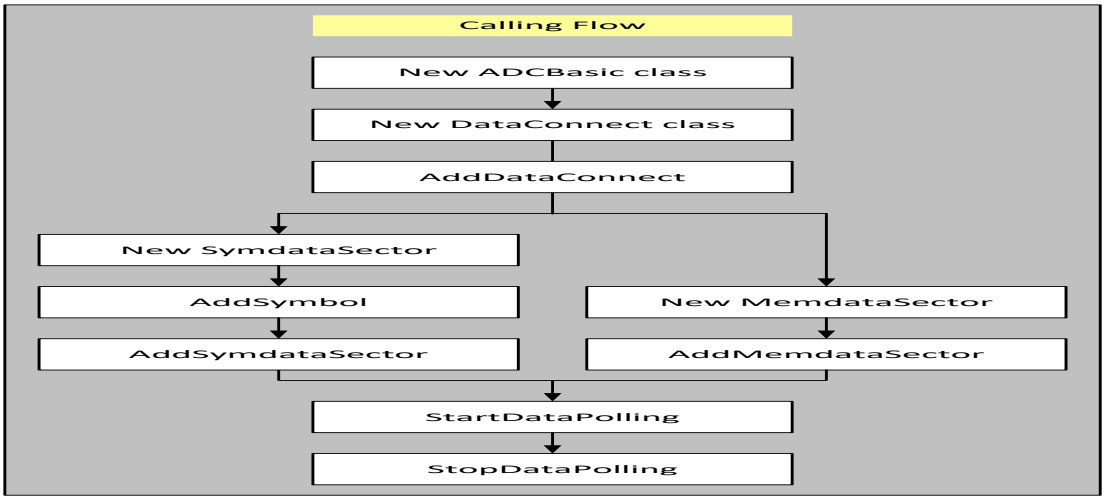
2.4. Data connection and sectors

The following diagram describes the relationship between connections and sectors:

User creates a data connection to connect to the controller using the controller IP address. Each data connection contains several symbol sectors and memory sectors. Every symbol sector contains more than one symbol. Every memory sector defines the starting memory address and the total number of %M area (flag memory). After completing the connection and sector creation, call **ADCBasic.StartDataPolling**, the ADC will periodically take values from the controller. User can write values back to controller by calling **DataConnect.WriteSymbolValue** or **DataConnect.WriteSingleMemoryValue** (**DataConnect.WriteMultipleMemoryValue**). The following sections describe the detailed API.

Note: Asynchronous Communication Mechanism
The ADC library performs symbol and memory access through an asynchronous polling mechanism. When ADCBasic.StartDataPolling() is called, a background thread continuously polls all configured connections and caches the symbol and memory data.

1. ***Read APIs (e.g., DataConnect.ReadSymbolValue, DataConnect.ReadSingleMemoryValue) return the latest cached values obtained from the last polling cycle rather than requesting immediate data from the controller.***
2. ***Write APIs (e.g., DataConnect.WriteSymbolValue, DataConnect.WriteSingleMemoryValue, DataConnect.WriteMultipleMemoryValue) queue write requests which are sent by the background thread asynchronously. Their completion is notified through corresponding callback functions (OnSymdataWriteCompletedEvent, OnMemdataWriteCompletedEvent).***
3. *A return value of ADC_SUCCESS means the operation request was successfully accepted by the ADC library, **not** that the data has already been applied on the PLC side.*
4. *Applications should use the callback functions to confirm completion or handle timing-sensitive logic.*



If you want to access the local flag memory (must be the same device as the CODESYS RTE), you can call read/write APIs (ADCBasic.ReadSingleMemoryValue_Local / ADCBasic.ReadMultipleMemoryValue_Local / ADCBasic.WriteSingleMemoryValue_Local / ADCBasic.WriteMultipleMemoryValue_Local) directly without create data connection and sector. And after the initialization is successful, it can be accessed directly without calling ADCBasic.StartDataPolling.

2.5. Synchronous Communication Mechanism

For synchronous access, first call new ADCBasic() and ADCBasic.AddDataConnect() to register connection settings. Optionally call DataConnect.SetMemConnectPort() before establishing the connection, then call DataConnect.ConnectSync().

After the connection is established, applications can call DataConnect.GetSymbolList(), DataConnect.ReadSymbolValueSync(), DataConnect.WriteSymbolValueSync(), DataConnect.ReadMemoryValueSync(), and DataConnect.WriteMemoryValueSync() directly. Symbol and Memory sectors and ADCBasic.StartDataPolling() are not required for this usage flow.

Call DataConnect.DisconnectSync(), ADCAPI.ADC_ReleaseADCResource(), and ADCAPI.ADC_Terminate() when finished. DataConnect.ConnectSync() establishes both PLCHandler and Modbus TCP connections; both must succeed.

Synchronous Symbol access can share a connected PLCHandler with polling. Synchronous Memory read/write cannot be called while polling is running and return ADC_THREAD_RUNNING in that state. The synchronous Modbus client is managed separately from the polling Modbus client.

Synchronous operations may wait for communication completion or timeout. A write timeout does not guarantee that the controller did not receive the write request.

3. ADC common Class

3.1. ADCBasic

public ADCBasic();

Purpose:

The constructor of ADCBasic class. To initialize the ADC library resource.

Parameters:

None.

3.2. AddDataConnect

```
public ADC_ErrorCode AddDataConnect(
        DataConnect obj
);
```

Purpose:

Add the specified data connection object into the polling list. This function has to be called before calling StartDataPolling().

Parameters:

obj = The specified data connection object.

Return:

ADC_SUCCESS, add the DataConnect succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_RUNNING, the polling thread is running.

ADC_OBJECT_DUPLICATED, the connection name is duplicated.

3.3. GetVersion

public uint GetVersion();

Purpose:
Get the version number of the ADC library.

Parameters:
None.

Return:
The version number. In hexadecimal format, for example 0x03020105, it means the version is V3.2.1.5.

3.4. SetADCPriority

```
public ADC_ErrorCode SetADCPriority(
    ADC_Priority priority
);
```

Purpose:
Set the priority of the ADC connection running thread.
Before calling this function, the data connection has to be successfully started.

Parameters:
priority = Specifies the priority value for the ADC thread. Reference the ADC_Priority enumeration.
This parameter can be one of the following values

- IDLE
- LOWEST
- BELOW_NORMAL
- NORMAL
- ABOVE_NORMAL

- HIGHEST
- TIME_CRITICAL

Return:

ADC_SUCCESS, set the ADC thread priority succeeded.

ADC_SETPRIO_FAILED, failed to set the thread priority.

3.5. StartDataPolling

public ADC_ErrorCode StartDataPolling();

Purpose:

Start the connections. After calling with ADC_SUCCESS returned, the ADC threads will start polling all controllers and their symbols/memory data.

Parameters:

None

Return:

ADC_SUCCESS, start all the DataConnect succeeded.

ADC_THREAD_RUNNING, the connections are already started

ADC_CREATE_THREAD_FAILED, failed to create the connection.

3.6. StopDataPolling

public ADC_ErrorCode StopDataPolling();

Purpose:

Stop the connections. After calling this function, the ADC thread will terminate and all polling will stop.

Parameters:

None

Return:

ADC_SUCCESS, stop all the DataConnect succeeded.

3.7. ShowConnection

public ADC_ErrorCode ShowConnection();

Purpose:

Show the current connections configuration.

Parameters:

None

Return:

ADC_SUCCESS, show the configurations succeeded.

3.8. ReadSingleMemoryValue_Local

```
public ADC_ErrorCode ReadSingleMemoryValue_Local(
    string i_szMemStartAddress,
    out byte[] byData
);
```

Purpose:

Read the specified memory data sector value from local flag memory. This function is only valid on the controller with Advantech CODESYS RTE installed.

Parameters:

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

byData = The variable of the memory data to be read. The value expressed in a byte array.

Return:

ADC_SUCCESS, read the memory data value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid
ADC_HANDLE_INVALID, failed to open local flag memory.

3.9. ReadMultipleMemoryValue_Local

```
public ADC_ErrorCode ReadMultipleMemoryValue_Local(
        string i_szMemStartAddress,
        int iTotal,
        out byte[] byData
);
```

Purpose:
Read the specified memory data sector value from local flag memory. This function is only valid on the controller with Advantech CODESYS RTE installed.

Parameters:
i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.
i_iTotal = The total of the memory item.
byData = The variable of the memory data to be read. The value expressed in a byte array.

Return:
ADC_SUCCESS, read the memory data value succeeded.
ADC_PARAMETER_INVALID, the input parameter is invalid
ADC_HANDLE_INVALID, failed to open local flag memory.

3.10. ReadMultipleMemoryValue_Local<T>

```
public ADC_ErrorCode ReadMemoryValue_Local<T>(
        string i_szMemStartAddress,
        out T Value
) where T : struct;
```

Purpose:

Read one typed value from local shared Memory. The data length is calculated from T and the Memory address unit.

Parameters:

i_szMemStartAddress = Local Memory start address (for example, %MW0).

Value = Output primitive or fixed-layout structure.

Note: Only valid on a device with Advantech CODESYS RTE local shared Memory. Supported address units are %MB, %MW, %MD and %ML.

Note: The marshaled size of T must be divisible by the selected address-unit size.

Return:

ADC_SUCCESS, the typed value was read from local shared Memory.

ADC_PARAMETER_INVALID, the Memory address or requested type is invalid.

ADC_SIZE_MISMATCH, the marshaled size of T does not match the Memory address-unit requirements.

ADC_HANDLE_INVALID, the local shared Memory could not be opened.

3.11. WriteSingleMemoryValue_Local

```
public ADC_ErrorCode WriteSingleMemoryValue_Local(
        string i_szMemStartAddress,
        byte[] byData
);
```

Purpose:

Write the specified memory data to local flag memory. This function is only valid on the controller with Advantech CODESYS RTE installed.

Parameters:

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

byData = The variable of the memory data to be write. The value expressed in a byte array.

Return:

ADC_SUCCESS, write the memory data value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_HANDLE_INVALID, failed to open local flag memory.

3.12. WriteMultipleMemoryValue_Local

```
public ADC_ErrorCode WriteMultipleMemoryValue_Local(
        string i_szMemStartAddress,
        int iTotal,
        byte[] byData
);
```

Purpose:

Write the specified memory data to local flag memory. This function is only valid on the controller with Advantech CODESYS RTE installed.

Parameters:

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

i_iTotal = The total of the memory item.

byData = The variable of the memory data to be write. The value expressed in a byte array.

Return:

ADC_SUCCESS, write the memory data value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_HANDLE_INVALID, failed to open local flag memory.

3.13. WriteMultipleMemoryValue_Local<T>

public ADC_ErrorCode WriteMemoryValue_Local<T>(
    string i_szMemStartAddress,
    T Value
)where T : struct;

Purpose:

Write one typed value to local shared Memory. The wrapper converts T into a byte buffer automatically.

Parameters:

i_szMemStartAddress = Local Memory start address.

Value = Primitive or fixed-layout structure to write.

Note: Same device requirement and Memory address-unit rules as ReadMemoryValue_Local<T>().

Return:

ADC_SUCCESS, the typed value was written to local shared Memory.

ADC_PARAMETER_INVALID, the Memory address or requested type is invalid.

ADC_SIZE_MISMATCH, the marshaled size of T does not match the Memory address-unit requirements.

ADC_HANDLE_INVALID, the local shared Memory could not be opened.

4. ADC DataConnect Class

4.1. DataConnect

```
public DataConnect(
        string i_szConnectName,
        string i_szIp
);
public DataConnect(
        string i_szConnectName,
        string i_szIp,
        string i_szUserName,
        string i_szPassword
);
```

Purpose:

The constructor of DataConnect class. After configuring the connection, call AddDataConnect to add this item to the polling list.

Parameters:

i_szConnectName = The connection name

i_szIp = IP address of the Controller.

i_szUserName = Username for PLC user management.

i_szPassword = Password for PLC user management.

i_iScanInterval = The scan interval for data reading.

i_iConnectTimeout = The connection timeout.

i_iTransactTimeout = The transaction timeout.

Return:

None

4.2. SetMemConnectPort

public ADC_ErrorCode SetMemConnectPort(UInt16 i_usPort);

Purpose:

Configure the Modbus TCP Memory port for this DataConnect object.

Parameters:

i_usPort = Port number (1–65535); default is 503.

Note: Call after ADCBasic.AddDataConnect() and before ConnectSync() or StartDataPolling().

Return:

ADC_SUCCESS, the Modbus TCP port was configured.

ADC_PARAMETER_INVALID, the port number or input parameter is invalid.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_MEM_CONNECTED, the Memory connection is active or polling is running.

4.3. ConnectSync

public ADC_ErrorCode ConnectSync();

Purpose:

Establish the PLCHandler and synchronous Modbus TCP transports for the registered connection.

Note: Both transports must connect successfully. No Symbol/Memory sectors or polling thread are required.

Parameters:

None

Return:

ADC_SUCCESS, both PLCHandler and synchronous Modbus TCP connections have been established.

ADC_HANDLE_INVALID, the connection name is invalid or does not exist.

ADC_PLC_NOT_CONNECTED, the PLCHandler connection could not be established.
ADC_MEM_CONNECT_FAILED, the synchronous Modbus TCP connection could not be established.
Other connection-related ADC error codes may be returned.

4.4. DisconnectSync

public ADC_ErrorCode DisconnectSync();

Purpose:
Release the synchronous connection. A PLCHandler shared with polling remains active while polling still needs it.

Parameters:
None

Return:
ADC_SUCCESS, the synchronous connection has been released.
ADC_HANDLE_INVALID, the connection name is invalid or does not exist.
Other connection-related ADC error codes may be returned.

4.5. ConnectionCompleted

public event OnConnectCompletedEvent ConnectionCompleted;

Purpose:
The call back function when the connection to the Controller completed event occurred.

Parameters:
None

Return:
None

4.6. DisConnectionCompleted

public event OnDisconnectCompletedEvent DisConnectionCompleted;

Purpose:
The call back function when the connection to the Controller closed event occurred

Parameters:
None

Return:
None

4.7. AddSymdataSector

public ADC_ErrorCode AddSymdataSector(
    SymdataSector obj
);

Purpose:
Add the specified Symdata sector into an existing connection for polling data from the controller. This function has to be called before calling ADCBasic.StartDataPolling().

Parameters:
obj = The specified SymdataSector object.

Return:
ADC_SUCCESS, add the sector succeeded.
ADC_PARAMETER_INVALID, the input parameter is invalid
ADC_THREAD_RUNNING, the polling thread is running.
ADC_HANDLE_INVALID, the connection does not exist.
ADC_OBJECT_DUPLICATED, the sector name is duplicated.

4.8. AddMemdataSector

public ADC_ErrorCode AddMemdataSector(
    MemdataSector obj
);

Purpose:
Add the specified Memdata sector into an existing connection for polling data from the controller. This function has to be called before calling ADCBasic.StartDataPolling().

Parameters:
obj = The specified MemdataSector object.

Return:
ADC_SUCCESS, add the sector succeeded.
ADC_PARAMETER_INVALID, the input parameter is invalid
ADC_THREAD_RUNNING, the polling thread is running.
ADC_HANDLE_INVALID, the connection does not exist.
ADC_OBJECT_DUPLICATED, the sector name is duplicated.
ADC_EXCEED_SECTOR_SIZE, the total size of the memory item exceeds the sector size.

4.9. GetSymbolList

public ADC_ErrorCode GetSymbolList(out ADC_SYMBOL_INFO[] SymbolList);

Purpose:
Retrieve Symbol names, type names and byte sizes published by the connected PLC.

Parameters:
SymbolList = Output managed array of ADC_SYMBOL_INFO records; each item contains szName (string), szType (string), and ulSize (UInt32, bytes).
Note: The C# wrapper converts the native list to managed records and calls ADC_FreeSymbolList() internally; the caller does not manually free it.
Note: The connection must already have a PLCHandler transport.

Return:

ADC_SUCCESS, the Symbol list was retrieved; an empty list may be returned.

ADC_PARAMETER_INVALID, an input or output argument is invalid.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_PLC_NOT_CONNECTED, the PLCHandler connection is unavailable.

ADC_PLC_SYMBOL_LOAD_FAILED, loading Symbols from the PLC failed.

ADC_MEMALLOC_FAILED, allocation failed.

Other PLCHandler-related ADC error codes may be returned.

4.9.1. ADC_SYMBOL_INFO

```
public struct ADC_SYMBOL_INFO
{
    public string szName;
    public string szType;
    public UInt32 ulSize;
}
```

Members:

szName = The Symbol name.

szType = The Symbol type name.

ulSize = The Symbol size in bytes.

Note: Unlike C++, C# callers do not need to call ADC_FreeSymbolList() after GetSymbolList().

4.10. ReadSymbolValue

```
public ADC_ErrorCode ReadSymbolValue(
    string i_szSymbolName,
    out byte[] byData
);
```

Purpose:

Read the specified symbol value from the connection. This function has to be called after calling ADCBasic.StartDataPolling().

Parameters:

i_szSymbolName = The symbol name.

byData = The variable of the symbol data to be read. The value expressed in a byte array.

Return:

ADC_SUCCESS, read the symbol value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_STOP, the polling thread is stop.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_OBJECT_NOT_EXIST, the symbol is not created in the sector.

ADC_PLC_SYMBOL_VALUE_NOT_READY, the value has not been retrieved

ADC_PLC_SYMBOL_SIZE_NOT_MATCH, the variable size is less than the symbol size.

ADC_PLC_NOT_CONNECTED, PLC is not connected.

ADC_PLC_NO_SYMBOLS, no symbols are available or symbol mismatch because of change project.

4.11. ReadSymbolValue<T>

```
public ADC_ErrorCode ReadSymbolValue<T>(
        string i_szSymbolName,
        out T Value
) where T : struct;
```

Purpose:

Read a typed Symbol value from the asynchronous polling cache.

Parameters:

i_szSymbolName = Published Symbol name, already added to the polling sector.

Value = Output primitive or fixed-layout value.

Note: Requires StartDataPolling(). The wrapper calculates the buffer size from T and checks the result size.

Return:

ADC_SUCCESS, the operation completed successfully.

ADC_PARAMETER_INVALID, an input parameter is invalid.

ADC_SIZE_MISMATCH, the received Symbol data size does not match the marshaled size of T.

ADC_THREAD_STOP, the polling thread is stopped.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_OBJECT_NOT_EXIST, the requested Symbol or Memory sector does not exist.

ADC_PLC_SYMBOL_VALUE_NOT_READY, the first Symbol polling read has not yet completed.

ADC_PLC_SYMBOL_SIZE_NOT_MATCH, the supplied variable buffer is smaller than the actual Symbol size.

ADC_PLC_NOT_CONNECTED, the PLCHandler is not connected.

ADC_PLC_NO_SYMBOLS, no Symbols are available or the project Symbol information is inconsistent.

Other PLCHandler-related ADC error codes may be returned.

4.12. WriteSymbolValue

```
public ADC_ErrorCode WriteSymbolValue(
        string i_szSymbolName,
        byte[] byData
);
public ADC_ErrorCode WriteSymbolValue(
        string i_szSymbolName,
        byte[] byData,
        OnSymdataWriteCompletedEvent evtHandler
);
```

Purpose:

Write the specified symbol value to the controller. The symbol to be written can be written successfully without adding to the sector. This function has to be called after calling ADCBasic.StartDataPolling().

Parameters:

i_szSymbolName = The symbol name.

byData = The variable of the symbol data to be write. The value expressed in a byte array.

evtHandler = The callback function that will be called when writing symbol data completed.

Return:

ADC_SUCCESS, write the symbol value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_STOP, the polling thread is stop.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_OBJECT_NOT_EXIST, the symbol does not exist.

ADC_PLC_NOT_CONNECTED, PLC is not connected.

ADC_PLC_NO_SYMBOLS, no symbols are available or symbol mismatch because of change project.

4.13. WriteSymbolValue<T>

```
public ADC_ErrorCode WriteSymbolValue<T>(
        string i_szSymbolName,
        T Value
) where T : struct
public ADC_ErrorCode WriteSymbolValue<T>(
        string i_szSymbolName,
        T Value,
        OnSymdataWriteCompletedEvent evtHandler
) where T : struct;
```

Purpose:

Queue an asynchronous write of a typed Symbol value.

Parameters:

i_szSymbolName = Published Symbol name.

Value = Typed value to write.

evtHandler = Write-completion callback (may be null).

Note: Use the callback to confirm actual completion. The wrapper marshals T to a byte array.

Return:

ADC_SUCCESS, the operation completed successfully.

ADC_PARAMETER_INVALID, an input parameter is invalid.

ADC_THREAD_STOP, the polling thread is stopped.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_OBJECT_NOT_EXIST, the requested Symbol or Memory sector does not exist.

ADC_PLC_NOT_CONNECTED, the PLCHandler is not connected.

ADC_PLC_NO_SYMBOLS, no Symbols are available or the project Symbol information is inconsistent.

Other PLCHandler-related ADC error codes may be returned.

4.14. ReadSymbolValueSync

```
public ADC_ErrorCode ReadSymbolValueSync(
        string i_szSymbolName,
        int i_iDataSize,
        out byte[] byData
);
```

Purpose:

Synchronously read a Symbol directly from the controller into a byte array.

Parameters:

i_szSymbolName = Published Symbol name.

i_iDataSize = Buffer size in bytes, at least the actual Symbol size.

byData = Output byte array if successful.

Note: Requires ConnectSync(); no polling sector required.

Return:

ADC_SUCCESS, the complete Symbol value was read.

ADC_PARAMETER_INVALID, an input parameter is invalid.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_PLC_NOT_CONNECTED, the PLCHandler is not connected.

ADC_OBJECT_NOT_EXIST, the requested Symbol or Memory sector does not exist.

ADC_BUFFER_TOO_SMALL, the output buffer is smaller than the Symbol size.
ADC_INCOMPLETE_DATA, the returned data is incomplete.
ADC_WRONG_CONTEXT, the API was called from an unsupported communication thread.
Other PLCHandler-related ADC error codes may be returned.

4.15. ReadSymbolValueSync<T>

```
public ADC_ErrorCode ReadSymbolValueSync<T>(
    string i_szSymbolName,
    out T Value
) where T : struct;
```

Purpose:
Synchronously read and convert a Symbol into the specified managed value type.

Parameters:
i_szSymbolName = Published Symbol name.
Value = Output typed value.
Note: The native byte size is derived from T. The C# struct layout must exactly match the PLC Symbol.

Return:
ADC_SUCCESS, the complete Symbol value was read.
ADC_PARAMETER_INVALID, an input parameter is invalid.
ADC_HANDLE_INVALID, the connection does not exist.
ADC_PLC_NOT_CONNECTED, the PLCHandler is not connected.
ADC_OBJECT_NOT_EXIST, the requested Symbol or Memory sector does not exist.
ADC_BUFFER_TOO_SMALL, the output buffer is smaller than the Symbol size.
ADC_INCOMPLETE_DATA, the returned data is incomplete.
ADC_WRONG_CONTEXT, the API was called from an unsupported communication thread.
ADC_SIZE_MISMATCH, the received Symbol data size does not match the marshaled size of T.
Other PLCHandler-related ADC error codes may be returned.

4.16. WriteSymbolValueSync

```
public ADC_ErrorCode WriteSymbolValueSync(
        string i_szSymbolName,
        byte[] byData
);
```

Purpose:

Synchronously write an entire Symbol buffer and return after the operation completes.

Parameters:

i_szSymbolName = Published Symbol name.

byData = Complete Symbol data; length must match the Symbol size.

Note: Requires ConnectSync().

Return:

ADC_SUCCESS, the Symbol write operation completed successfully.

ADC_PARAMETER_INVALID, an input parameter is invalid.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_PLC_NOT_CONNECTED, the PLCHandler is not connected.

ADC_OBJECT_NOT_EXIST, the requested Symbol or Memory sector does not exist.

ADC_SIZE_MISMATCH, the input data size does not match the Symbol size.

ADC_WRONG_CONTEXT, the API was called from an unsupported communication thread.

Other PLCHandler-related ADC error codes may be returned.

4.17. WriteSymbolValueSync<T>

```
public ADC_ErrorCode WriteSymbolValueSync<T>(
        string i_szSymbolName,
        T Value
) where T : struct;
```

Purpose:

Synchronously write one typed Symbol, converting T to a native byte buffer.

Parameters:

i_szSymbolName = Published Symbol name.

Value = Primitive or fixed-layout structure to write.

Note: The native size/layout must match the PLC Symbol exactly.

Return:

ADC_SUCCESS, the Symbol write operation completed successfully.

ADC_PARAMETER_INVALID, an input parameter is invalid.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_PLC_NOT_CONNECTED, the PLCHandler is not connected.

ADC_OBJECT_NOT_EXIST, the requested Symbol or Memory sector does not exist.

ADC_SIZE_MISMATCH, the size of the typed value does not match the PLC Symbol size.

ADC_WRONG_CONTEXT, the API was called from an unsupported communication thread.

Other PLCHandler-related ADC error codes may be returned.

4.18. ReadSingleMemoryValue

```
public ADC_ErrorCode ReadSingleMemoryValue(
        string i_szMemStartAddress,
        out byte[] byData
);
```

Purpose:

Read the specified memory data sector value from the connection. This function has to be called after calling ADCBasic.StartDataPolling().

Parameters:

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

byData = The variable of the memory data to be read. The value expressed in a byte array.

Return:

ADC_SUCCESS, read the memory data value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_STOP, the polling thread is stop.
ADC_HANDLE_INVALID, the connection does not exist.
ADC_MEM_SOCKET_CLOSED, the connection does not connect.
ADC_OBJECT_NOT_EXIST, the memory data is not created in the sector.

4.19. ReadMultipleMemoryValue

```
public ADC_ErrorCode ReadMultipleMemoryValue(
        string i_szMemStartAddress,
        int iTotal,
        out byte[] byData
);
```

Purpose:
Read the specified memory data sector value from the connection. This function has to be called after calling ADCBasic.StartDataPolling().

Parameters:
i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.
i_iTotal = The total of the memory item.
byData = The variable of the memory data to be read. The value expressed in a byte array.

Return:
ADC_SUCCESS, read the memory data value succeeded.
ADC_PARAMETER_INVALID, the input parameter is invalid
ADC_THREAD_STOP, the polling thread is stop.
ADC_HANDLE_INVALID, the connection does not exist.
ADC_MEM_SOCKET_CLOSED, the connection does not connect.
ADC_OBJECT_NOT_EXIST, the memory data is not created in the sector.

4.20. ReadMultipleMemoryValue<T>

```
public ADC_ErrorCode ReadMemoryValue<T>(
        string i_szMemStartAddress,
        out T Value
) where T : struct;
```

Purpose:

Read a typed Memory value from the asynchronous polling cache.

Parameters:

i_szMemStartAddress = Memory start address.

Value = Converted typed output.

Note: Memory region must be configured in a polling sector. Size of T must be divisible by the address unit.

Return:

ADC_SUCCESS, the operation completed successfully.

ADC_PARAMETER_INVALID, an input parameter is invalid.

ADC_SIZE_MISMATCH, the size of T is incompatible with the selected Memory address unit or returned data size.

ADC_THREAD_STOP, the polling thread is stopped.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_MEM_SOCKET_CLOSED, the Memory connection is closed.

ADC_OBJECT_NOT_EXIST, the requested Symbol or Memory sector does not exist.

Other Memory transport or Modbus exception codes may be returned.

4.21. WriteSingleMemoryValue

```
public ADC_ErrorCode WriteSingleMemoryValue(
        string i_szMemStartAddress,
        byte[] byData
);

public ADC_ErrorCode WriteSingleMemoryValue(
        string i_szMemStartAddress,
        byte[] byData,
        OnMemdataWriteCompletedEvent evtHandler
);
```

Purpose:

Write the value to the specified memory area to the controller. The memory area to be written can be written successfully without adding to the sector. This function has to be called after calling ADCBasic.StartDataPolling().

Parameters:

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

byData = The variable of the memory data to be write. The value expressed in a byte array.

evtHandler = The callback function that will be called when writing memory data completed.

Return:

ADC_SUCCESS, write the memory data value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_STOP, the polling thread is stop.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_MEM_SOCKET_CLOSED, the connection does not connect.

ADC_MEM_THREAD_BUSY, the previous write memory operation was not completed.

4.22. WriteMultipleMemoryValue

```
public ADC_ErrorCode WriteMultipleMemoryValue(
        string i_szMemStartAddress,
        int iTotal,
        byte[] byData
);

public ADC_ErrorCode WriteMultipleMemoryValue(
        string i_szMemStartAddress,
        int iTotal,
        byte[] byData,
        OnMemdataWriteCompletedEvent evtHandler
);
```

Purpose:

Write the value to the specified memory area to the controller. The memory area to be written can be written successfully without adding to the sector. This function has to be called after calling ADCBasic.StartDataPolling().

Parameters:

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.
i_iTotal = The total of the memory item. The maximum number of %MB items is **253**. The maximum number of %MW items is **127**. The maximum number of %MD items is **63**.
byData = The variable of the memory data to be write. The value expressed in a byte array.
evtHandler = The callback function that will be called when writing memory data completed.

Return:

ADC_SUCCESS, write the memory data value succeeded.
ADC_PARAMETER_INVALID, the input parameter is invalid
ADC_THREAD_STOP, the polling thread is stop.
ADC_HANDLE_INVALID, the connection does not exist.
ADC_MEM_SOCKET_CLOSED, the connection does not connect.
ADC_MEM_THREAD_BUSY, the previous write memory operation was not completed.

ADC_EXCEED_SECTOR_SIZE, the total size of the memory item exceeds the sector size.

4.23. WriteMultipleMemoryValue <T>

```
public ADC_ErrorCode WriteMultipleMemoryValue <T>(
        string i_szMemStartAddress,
        T Value
) where T : struct;

public ADC_ErrorCode WriteMultipleMemoryValue <T>(
        string i_szMemStartAddress,
        T Value,
        OnMemdataWriteCompletedEvent evtHandler
) where T : struct;
```

Purpose:

Queue an asynchronous Memory write using one typed value.

Parameters:

i_szMemStartAddress = Memory start address.

Value = Value to write.

evtHandler = Callback notified when the write completes.

Note: The wrapper determines the Memory item count based on the value size and the address unit.

Return:

ADC_SUCCESS, the operation completed successfully.

ADC_PARAMETER_INVALID, an input parameter is invalid.

ADC_SIZE_MISMATCH, the size of T is incompatible with the selected Memory address unit or returned data size.

ADC_THREAD_STOP, the polling thread is stopped.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_MEM_SOCKET_CLOSED, the Memory connection is closed.

ADC_MEM_THREAD_BUSY, a previous asynchronous Memory write has not completed.

ADC_EXCEED_SECTOR_SIZE, the requested Memory item count exceeds the sector size.

Other Memory transport or Modbus exception codes may be returned.

4.24. ReadMemoryValueSync

```
public ADC_ErrorCode ReadMemoryValueSync(
        string i_szMemStartAddress,
        int i_iDataSize,
        out byte[] byData
);
```

Purpose:

Synchronously read the requested number of bytes from PLC Memory.

Parameters:

i_szMemStartAddress = CODESYS %MB, %MW, %MD or %ML address.

i_iDataSize = Requested length in bytes.

byData = Complete output byte array.

Note: ConnectSync() is required. Synchronous Memory calls are not available while Memory polling is running.

Note: Requests may be segmented by protocol or Memory-bank boundaries.

Return:

ADC_SUCCESS, the requested Memory data was read.

ADC_PARAMETER_INVALID, the Memory address, range, or another parameter is invalid.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_THREAD_RUNNING, Memory polling is running.

ADC_WRONG_CONTEXT, the API was called from the Modbus communication thread.

ADC_MEM_CONNECT_FAILED, the synchronous Modbus TCP connection is unavailable.

ADC_MEM_WAIT_TIMEOUT, the socket operation timed out.

ADC_INCOMPLETE_DATA, the response does not contain the complete required data.

Other Memory transport or Modbus exception codes may be returned.

4.25. ReadMemoryValueSync<T>

public ADC_ErrorCode ReadMemoryValueSync<T>(
    string i_szMemStartAddress,
    out T Value
) where T : struct;

Purpose:

Synchronously read one typed Memory value. The requested byte count is derived from T.

Parameters:

i_szMemStartAddress = Memory start address.

Value = Converted output value.

Note: T must be a supported primitive or fixed-layout structure with an appropriate byte count for the address unit.

Return:

ADC_SUCCESS, the requested Memory data was read.

ADC_PARAMETER_INVALID, the Memory address, range, or another parameter is invalid.

ADC_SIZE_MISMATCH, the size of T is incompatible with the selected Memory address unit or returned data size.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_THREAD_RUNNING, Memory polling is running.

ADC_WRONG_CONTEXT, the API was called from the Modbus communication thread.

ADC_MEM_CONNECT_FAILED, the synchronous Modbus TCP connection is unavailable.

ADC_MEM_WAIT_TIMEOUT, the socket operation timed out.

ADC_INCOMPLETE_DATA, the response does not contain the complete required data.

Other Memory transport or Modbus exception codes may be returned.

4.26. WriteMemoryValueSync

```
public ADC_ErrorCode WriteMemoryValueSync(
        string i_szMemStartAddress,
        byte[] byData
);
```

Purpose:

Synchronously write a complete byte range to PLC Memory.

Parameters:

i_szMemStartAddress = CODESYS Memory start address.

byData = Input bytes, with length defining the write size.

Note: The operation may be segmented. Earlier segments may have been written even if a later segment fails.

Note: Cannot be used while Memory polling is running.

Return:

ADC_SUCCESS, the Memory write operation completed successfully.

ADC_PARAMETER_INVALID, the Memory address, range, or another parameter is invalid.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_THREAD_RUNNING, Memory polling is running.

ADC_WRONG_CONTEXT, the API was called from the Modbus communication thread.

ADC_MEM_CONNECT_FAILED, the synchronous Modbus TCP connection is unavailable.

ADC_MEM_WAIT_TIMEOUT, the socket operation timed out.

Other Memory transport or Modbus exception codes may be returned.

4.27. WriteMemoryValueSync<T>

public ADC_ErrorCode WriteMemoryValueSync<T>(
    string i_szMemStartAddress,
    T Value
) where T : struct;

Purpose:

Synchronously write one typed Memory value.

Parameters:

i_szMemStartAddress = Memory start address.

Value = Typed value to write.

Note: Size is derived from T; address-unit alignment must be valid.

Return:

ADC_SUCCESS, the Memory write operation completed successfully.

ADC_PARAMETER_INVALID, the Memory address, range, or another parameter is invalid.

ADC_SIZE_MISMATCH, the size of T is incompatible with the selected Memory address unit or returned data size.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_THREAD_RUNNING, Memory polling is running.

ADC_WRONG_CONTEXT, the API was called from the Modbus communication thread.

ADC_MEM_CONNECT_FAILED, the synchronous Modbus TCP connection is unavailable.

ADC_MEM_WAIT_TIMEOUT, the socket operation timed out.

Other Memory transport or Modbus exception codes may be returned.

5. ADC Symdata Sector Class

5.1. SymdataSector

```
public SymdataSector(
        string i_szSectorName
);
```

Purpose:

The constructor of SymdataSector class. After configuring the sector, call AddSymdataSector to add this item to the specified data connection.

Parameters:

i_szSectorName = The polling sector name.

Return:

None

5.2. SymdataReadCompleted

```
public event OnSymdataReadCompletedEvent SymdataReadCompleted;
```

Purpose:

The delegate call back function when the symbol data read completed event occurred.

Parameters:

None

Return:

None

5.3. AddSymbol

```
public void AddSymbol(
        string i_szSymbolName
);
```

Purpose:

Add a symbol name into an existing sector.

Parameters:

i_szSymbolName = The symbol name.

Return:

None

6. ADC Memdata Sector Class

6.1. MemdataSector

```
public MemdataSector(
        string i_szSectorName,
        string i_szMemStartAddress,
        int i_iTotal
);
```

Purpose:

The constructor of MemdataSector class. After configuring the sector, call AddMemdataSector to add this item to the specified data connection.

Parameters:

i_szSectorName = The polling sector name.

i_szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

i_iTotal = The total of the memory item. The maximum number of %MB items in a sector is **253**. The maximum number of %MW items in a sector is **127**. The maximum number of %MD items in a sector is **63**.

Return:

None

6.2. MemdataReadCompleted

public event OnMemdataReadCompletedEvent MemdataReadCompleted;

Purpose:

The delegate call back function when the memory data read completed event occurred.

Parameters:

None

Return:

None

7. ADC Conversion Tool Class

7.1. ByteArrayConvertToString

```
public static void ByteArrayConvertToString(
        byte[] byData,
        out string oValue
);
```

Purpose:

Convert byte array to string format variable.

Parameters:

byData = The value expressed in a byte array

oValue = The converted value expressed in a string type

Return:

None

7.2. ByteArrayConvertToUint16Array

```
public static void ByteArrayConvertToUint16Array(
        byte[] byData,
        out UInt16[] oValue
);
```

Purpose:

Convert byte array to UInt16 format array.

Parameters:

byData = The value expressed in a byte array

oValue = The converted value expressed in an UInt16 array

Return:

None

7.3. ByteArrayConvertToUint32Array

```
public static void ByteArrayConvertToUint32Array(
        byte[] byData,
        out UInt32[] oValue
);
```

Purpose:

Convert byte array to UInt32 format array.

Parameters:

byData = The value expressed in a byte array

oValue = The converted value expressed in an UInt32 array

Return:

None

7.4. ByteArrayConvertToUint64Array

```
public static void ByteArrayConvertToUint64Array(
        byte[] byData,
        out UInt64[] oValue
);
```

Purpose:

Convert byte array to UInt64 format array.

Parameters:

byData = The value expressed in a byte array

oValue = The converted value expressed in an UInt64 array

Return:

None

7.5. ByteArrayConvertToSbyteArray

```
public static void ByteArrayConvertToSbyteArray(
        byte[] byData,
        out sbyte[] oValue
);
```

Purpose:

Convert byte array to Sbyte format array.

Parameters:

byData = The value expressed in a byte array

oValue = The converted value expressed in a sbyte array

Return:

None

7.6. ByteArrayConvertToInt16Array

```
public static void ByteArrayConvertToInt16Array(
        byte[] byData,
        out Int16[] oValue
);
```

Purpose:

Convert byte array to Int16 format array.

Parameters:

byData = The value expressed in a byte array

oValue = The converted value expressed in an Int16 array

Return:

None

7.7. ByteArrayConvertToInt32Array

```
public static void ByteArrayConvertToInt32Array(
        byte[] byData,
        out Int32[] oValue
);
```

Purpose:

Convert byte array to Int32 format array.

Parameters:

byData = The value expressed in a byte array

oValue = The converted value expressed in an Int32 array

Return:

None

7.8. ByteArrayConvertToInt64Array

```
public static void ByteArrayConvertToInt64Array(
        byte[] byData,
        out Int64[] oValue
);
```

Purpose:

Convert byte array to Int64 format array.

Parameters:

byData = The value expressed in a byte array

oValue = The converted value expressed in an Int64 array

Return:

None

7.9. ByteArrayConvertToFloatArray

```
public static void ByteArrayConvertToFloatArray(
    byte[] byData,
    out float[] oValue
);
```

Purpose:

Convert byte array to float format array.

Parameters:

byData = The value expressed in a byte array

oValue = The converted value expressed in a float array

Return:

None

7.10. ByteArrayConvertToDoubleArray

```
public static void ByteArrayConvertToDoubleArray(
    byte[] byData,
    out double[] oValue
);
```

Purpose:

Convert byte array to double format array.

Parameters:

byData = The value expressed in a byte array

oValue = The converted value expressed in a double array

Return:

None

7.11. StringConvertToByteArray

```
public static void StringConvertToByteArray(
        string Value,
        out byte[] byData
);
```

Purpose:

Convert string array to byte format array. The size of byData is string length + 1. (include "\0")

For example:

- Value = abcd, output array is [97, 98, 99, 100, 0]

Parameters:

Value = The value expressed in a string type

byData = The converted value expressed in a byte array

Return:

None

7.12. StringConvertToByteArray

```
public static void StringConvertToByteArray(
        string Value,
        int iLen,
        out byte[] byData
);
```

Purpose:

Convert string array to byte format array. The size of byData is iLen (or iLen+1)

For example:

- Value = abcd, iLen = 4, output array is [97, 98, 99, 100, 0]
- Value = abcd, iLen = 10, output array is [97, 98, 99, 100, 0, 0, 0, 0, 0, 0]

Parameters:

Value = The value expressed in a string type.

iLen= Expected array size.

byData = The converted value expressed in a byte array.

Return:

None

7.13. Uint16ArrayConvertToByteArray

```
public static void Uint16ArrayConvertToByteArray(
    UInt16[] Value,
    out byte[] byData
);
```

Purpose:

Convert UInt16 array to byte format array.

Parameters:

Value = The value expressed in an UInt16 array

byData = The converted value expressed in a byte array

Return:

None

7.14. Uint32ArrayConvertToByteArray

```
public static void Uint32ArrayConvertToByteArray(
    UInt32[] Value,
    out byte[] byData
```

);

Purpose:

Convert UInt32 array to byte format array.

Parameters:

Value = The value expressed in an UInt32 array

byData = The converted value expressed in a byte array

Return:

None

7.15. Uint64ArrayConvertToByteArray

```
public static void Uint64ArrayConvertToByteArray(
    UInt64[] Value,
    out byte[] byData
);
```

Purpose:

Convert UInt64 array to byte format array.

Parameters:

Value = The value expressed in an UInt64 array

byData = The converted value expressed in a byte array

Return:

None

7.16. SbyteArrayConvertToByteArray

```
public static void SbyteArrayConvertToByteArray(
    sbyte[] Value,
    out byte[] byData
```

);

Purpose:

Convert sbyte array to byte format array.

Parameters:

Value = The value expressed in a sbyte array

byData = The converted value expressed in a byte array

Return:

None

7.17. Int16ArrayConvertToByteArray

```
public static void Int16ArrayConvertToByteArray(
    Int16[] Value,
    out byte[] byData
);
```

Purpose:

Convert Int16 array to byte format array.

Parameters:

Value = The value expressed in an Int16 array

byData = The converted value expressed in a byte array

Return:

None

7.18. Int32ArrayConvertToByteArray

```
public static void Int32ArrayConvertToByteArray(
    Int32[] Value,
    out byte[] byData
```

);

Purpose:

Convert Int32 array to byte format array.

Parameters:

Value = The value expressed in an Int32 array

byData = The converted value expressed in a byte array

Return:

None

7.19. Int64ArrayConvertToByteArray

```
public static void Int64ArrayConvertToByteArray(
    Int64[] Value,
    out byte[] byData
);
```

Purpose:

Convert Int64 array to byte format array.

Parameters:

Value = The value expressed in an Int64 array

byData = The converted value expressed in a byte array

Return:

None

7.20. FloatArrayConvertToByteArray

```
public static void FloatArrayConvertToByteArray(
    float[] Value,
    out byte[] byData
```

);

Purpose:

Convert float array to byte format array.

Parameters:

Value = The value expressed in a float array

byData = The converted value expressed in a byte array

Return:

None

7.21. DoubleArrayConvertToByteArray

```
public static void DoubleArrayConvertToByteArray (
    float[] Value,
    out byte[] byData
);
```

Purpose:

Convert double array to byte format array.

Parameters:

Value = The value expressed in a double array

byData = The converted value expressed in a byte array

Return:

None

8. ADC delegate functions

8.1. OnConnectCompletedEvent

```
public delegate void OnConnectCompletedEvent(
        Int32 i_lResult,
        string i_szConnectionName,
        IntPtr i_Param
);
```

Purpose:

The prototype of call back function when the connection to the Controller completed event occurred.

Parameters:

i_lResult = The result of the connection.

i_szConnectionName = The connection name

i_Param = The value is always NULL

Return:

None

8.2. OnDisconnectCompletedEvent

```
public delegate void OnDisconnectCompletedEvent(
        Int32 i_lResult,
        string i_szConnectionName,
        IntPtr i_Param
);
```

Purpose:

The prototype of call back function when the connection to the Controller closed event occurred.

Parameters:

i_lResult = The result of the connection.

i_szConnectionName = The connection name

i_Param = The value is always NULL

Return:

None

8.3. OnSymdataReadCompletedEvent

```
public delegate void OnSymdataReadCompletedEvent(
        Int32 i_lResult,
        string i_szSectorName,
        int i_iTotal,
        IntPtr i_Param
);
```

Purpose:

The prototype of call back function when the symbol data read completed event occurred.

Parameters:

i_lResult = The result of the reading.

i_ szSectorName = The polling sector name.

i_iTotal = The total of the symbol read.

i_Param = The value is always NULL.

Return:

None

8.4. OnMemdataReadCompletedEvent

```
public delegate void OnMemdataReadCompletedEvent(
    Int32 i_lResult,
    string i_szSectorName,
    int i_iTotal,
    IntPtr i_Param
);
```

Purpose:

The prototype of call back function when the memory data read completed event occurred.

Parameters:

i_lResult = The result of the reading.

i_szSectorName = The polling sector name.

i_iTotal = The total of the memory item read.

i_Param = The value is always NULL.

Return:

None

8.5. OnSymdataWriteCompletedEvent

```
public delegate void OnSymdataWriteCompletedEvent(
    Int32 i_lResult,
    IntPtr i_Param
);
```

Purpose:

The prototype of call back function when the symbol data write completed event occurred.

Parameters:

i_lResult = The result of the connection.

i_Param = The value is always NULL.

Return:

None

8.6. OnMemdataWriteCompletedEvent

```
public delegate void OnMemdataWriteCompletedEvent(
    Int32 i_lResult,
    IntPtr i_Param
);
```

Purpose:

The prototype of call back function when the memory data write completed event occurred.

Parameters:

i_lResult = The result of the writing.

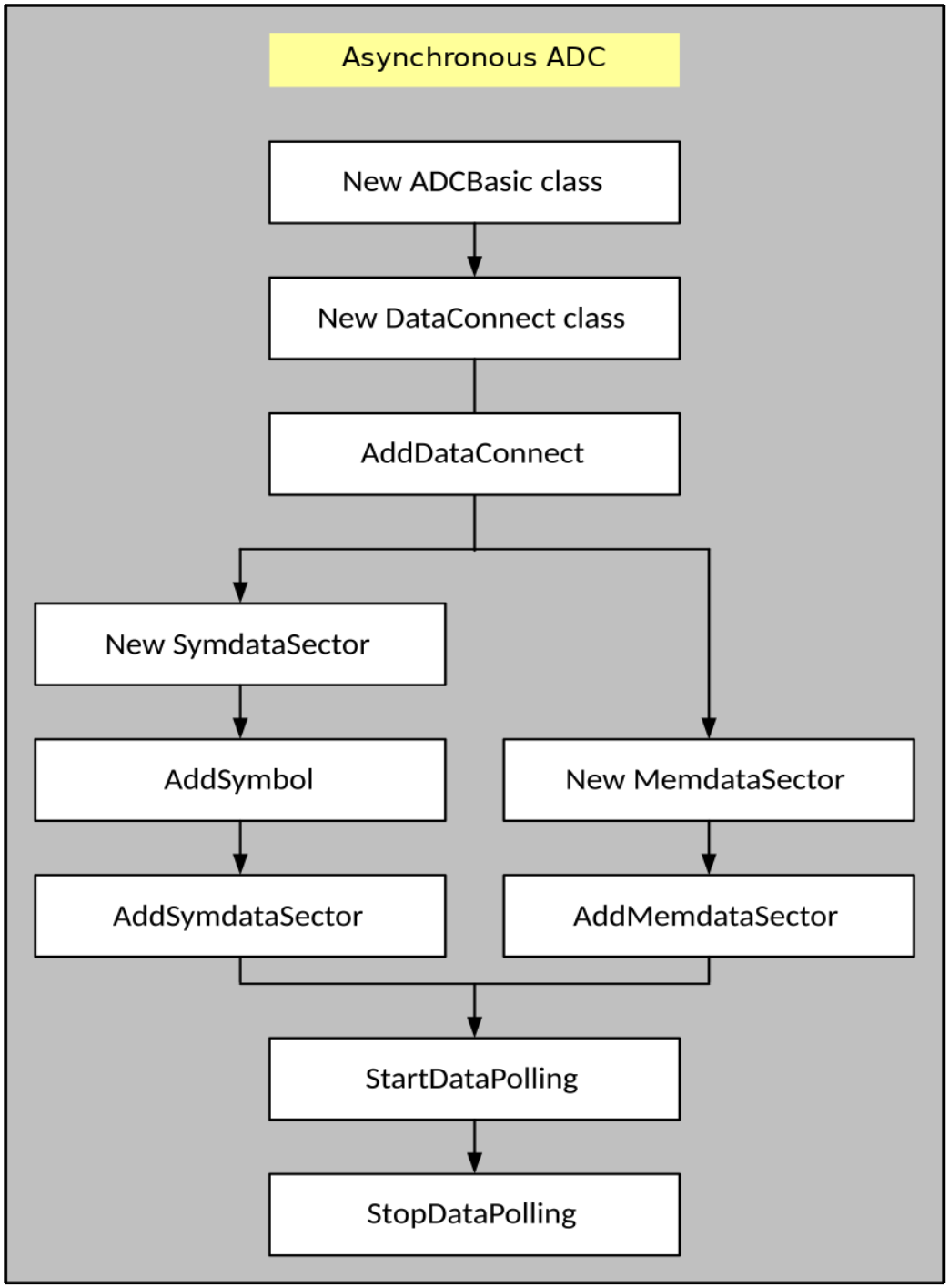
i_Param = The value is always NULL.

Return:

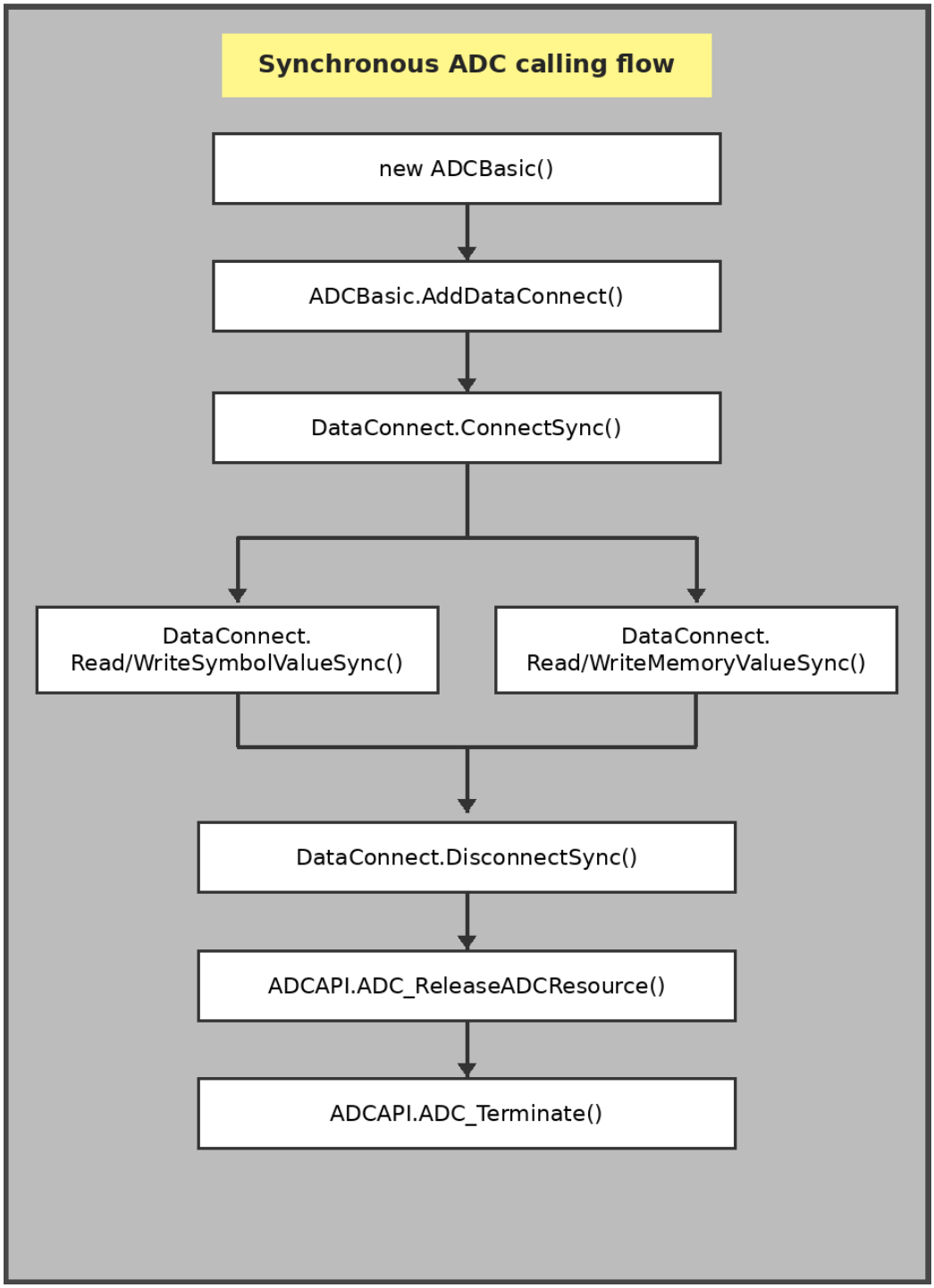
None

9. ADC calling flow

9.1. Asynchronous ADC calling flow



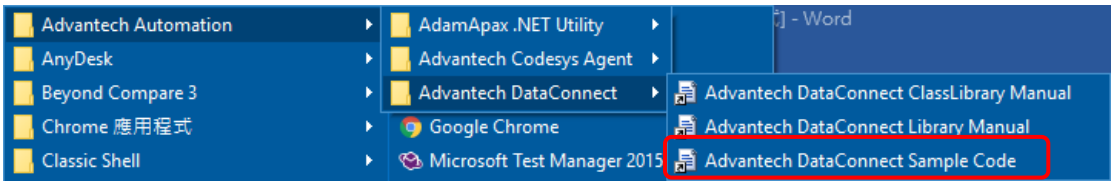
9.2. Synchronous ADC calling flow



The synchronous flow requires no Symbol or Memory sectors and does not call StartDataPolling(). The cleanup steps shown use the public low-level ADCAPI methods, which are called explicitly by the supplied C# samples.

10. ADC sample code

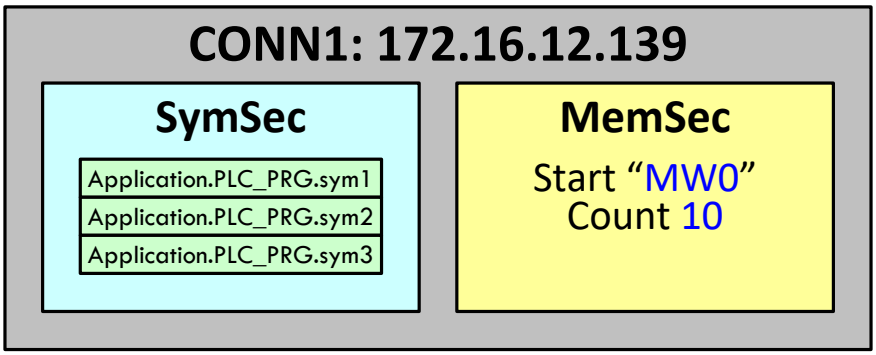
After installing the Advantech CODESYS Agent package, you can find the path of sample code and manual in the quick start: **Advantech Automation→Advantech DataConnect**



10.1. Basic example

Please reference the project **Example1** in **Example.sln**. This example shows how to use ADC API to access controller symbols and flag memory. The cooperating CODESYS project is **example.project**.

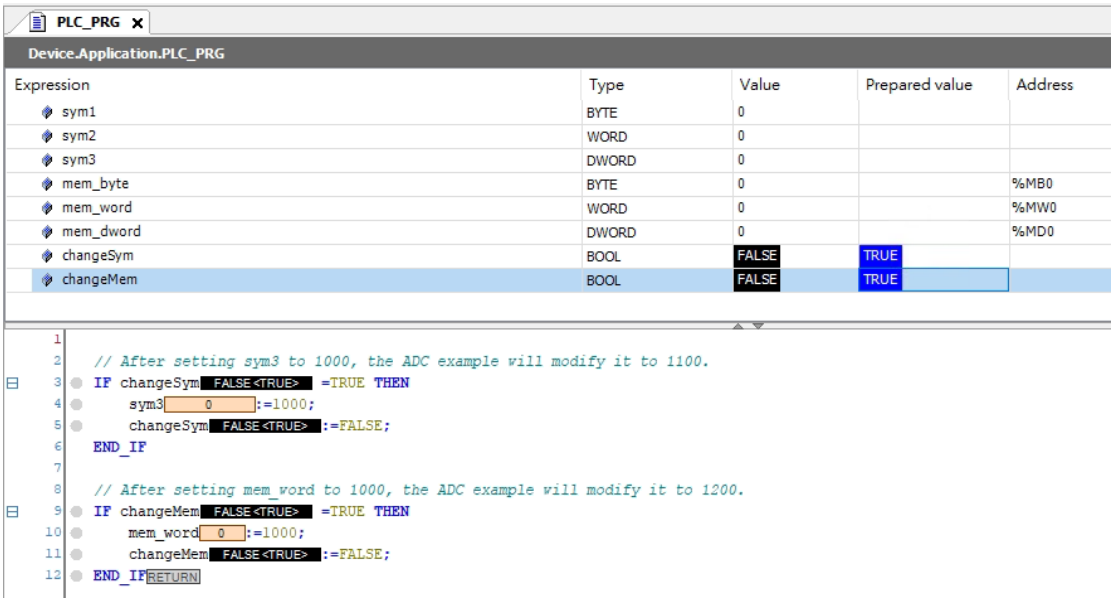
This example creates a new data connection named " **Connect1**". And then create a symbol sector named "**SymSec**" to " **Connect1**". Add 3 symbols to the sector. Then create a symbol sector named "**MemSec**" to " **Connect1**". Set the sector from **%MW0** ~ **%MW9**. As shown below.



After calling **StartDataPolling()**, the ADC thread will start polling all controllers and their symbol / memory data. In the callback read complete function, check whether sym3 is completed and the value is 1000, modify its value to 1100 and write it back to the controller.

Similarly, when reading% MW0 is completed and the value is 1000, modify the value to 1200 and write it back to the controller.

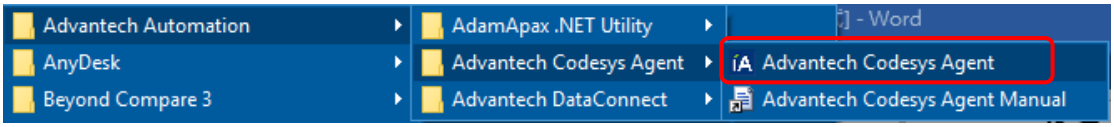
Back to the CODESYS project, we set **changeSym** and **changeMem** to **true**. Then, the ADC example will modify the value of **sym3** and **%MW0**.



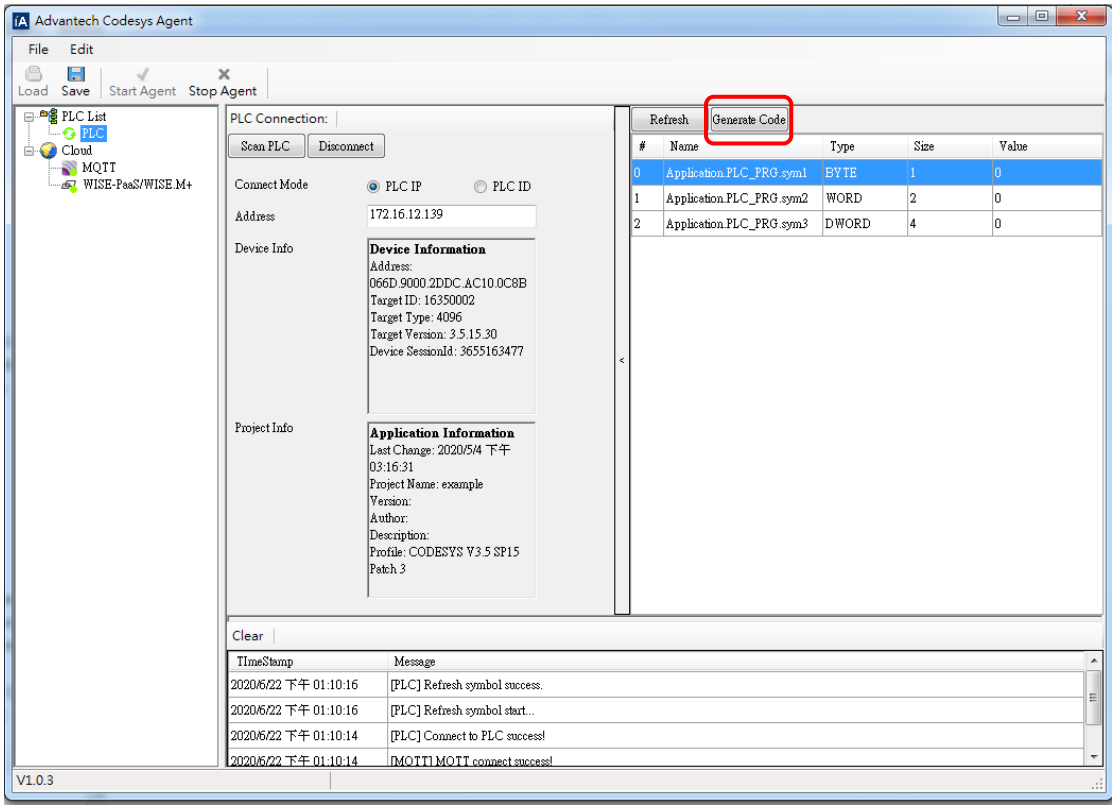
10.2. Symbol example generate

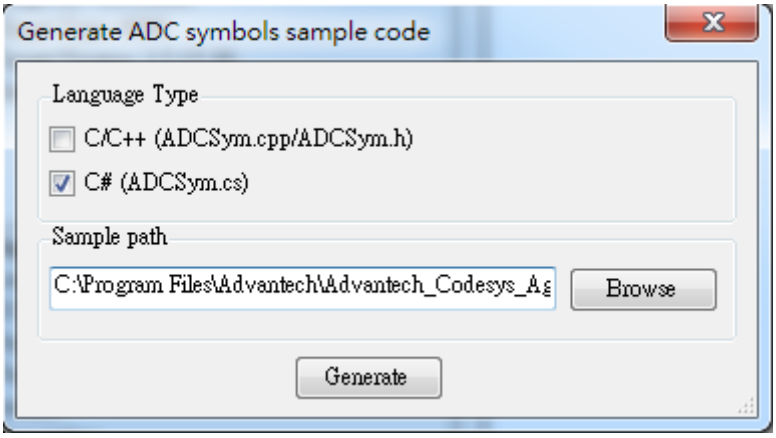
Please reference the project **Example2** in **Example.sln**. This example shows how to reference the automatically generated code. The cooperating CODESYS project is **example.project**.

Open the **Advantech Codesys Agent** in the quick start: **Advantech Automation→Advantech Codesys Agent.**

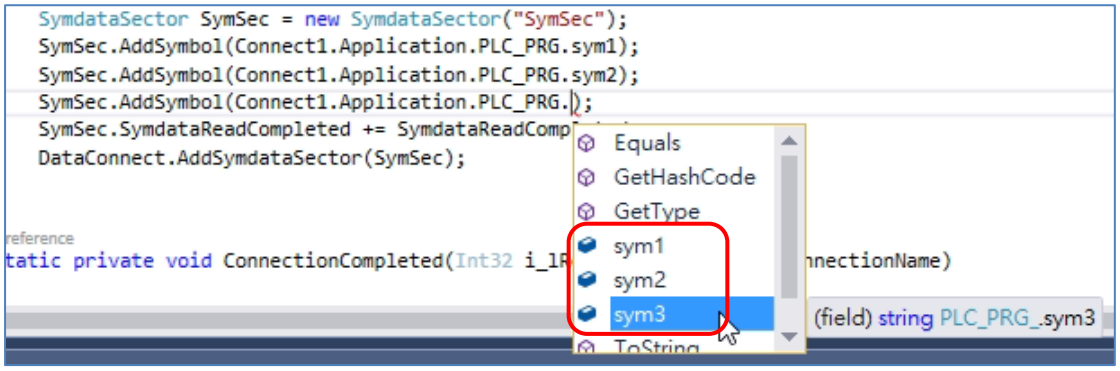


Add a PLC in PLC List and fill the PLC IP address. Click the **Start Agent** and the agent will automatically fetch the symbols and value from the controller. Click the **Generate Code** button and select the language type to save relevant files. It will automatically generate the **ADCSym.cs**.





The user only needs to copy these files to the path of the **Example2** folder. In addition to the easy creation of variables, it also provides the function of automatically listing symbol members.



10.3. Access local flag memory

Please reference the project **Example3** in **Example.sln**. This example shows how to use ADC API to access local flag memory.

If you want to access the local flag memory (must be the same device as the CODESYS RTE), you can call read/write APIs (**ADCBasic.ReadSingleMemoryValue_Local** / **ADCBasic.ReadMultipleMemoryValue_Local** / **ADCBasic.WriteSingleMemoryValue_Local** / **ADCBasic.WriteMultipleMemoryValue_Local**) directly without create data connection and sector. And after the initialization is successful, it can be accessed directly without calling **ADCBasic.StartDataPolling**.

This example first reads %MW0 from the local flag memory. If the result is successful, check if the value is 1000, and then write back 1200 to %MW0.

Then read the entire flag memory data and write them all to 0xff. (Need to define "ReadWriteAll")

10.4. Synchronous Access

Please reference the sample code Example4.cs. This example demonstrates synchronous Symbol and Memory read/write using byte arrays, including retrieving the Symbol list through DataConnect.GetSymbolList().

The example reads and writes BYTE, WORD and DWORD Symbols, as well as %MW0 (16-bit) and %MD1 (32-bit) Memory values. It also demonstrates setting the Memory connection port to 503 and releasing ADC resources.

Note: Configure the PLC IP, credentials and target Symbols for your device. This example actively overwrites writable Symbols of the supported types and the two specified Memory locations; run it only against a safe test project.

10.5. Synchronous Access Using Generic APIs

Please reference the sample code Example4_2.cs. This example demonstrates synchronous Symbol and Memory read/write using C# generic APIs.

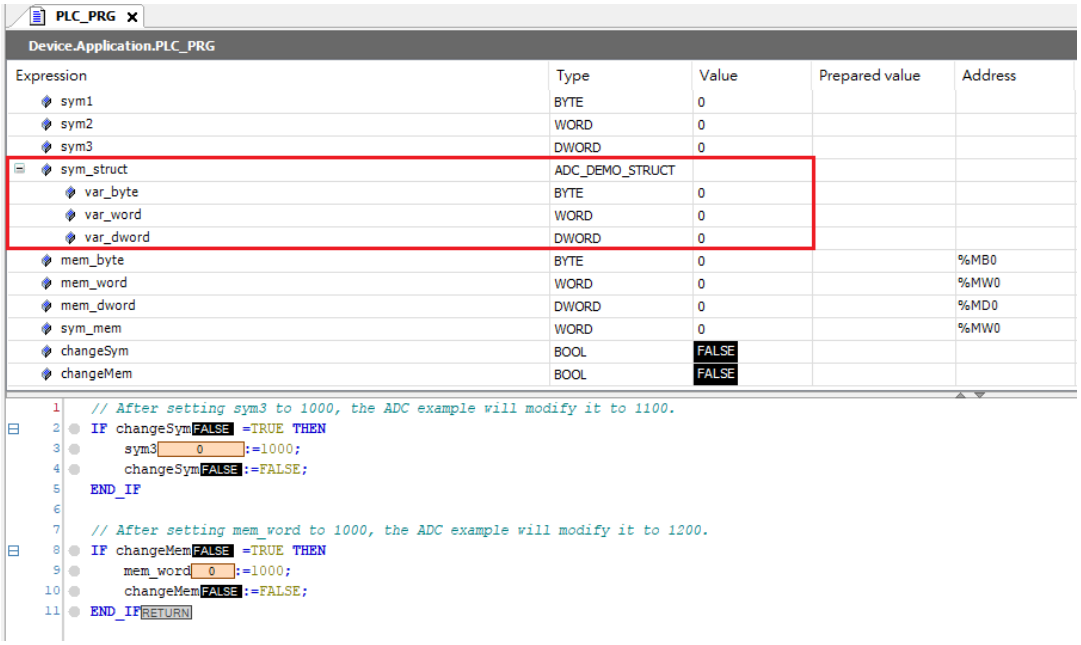
The example retrieves the Symbol list and reads and writes BYTE, WORD and DWORD Symbols as byte, ushort and uint values. It also reads and writes %MW0 (ushort) and %MD1 (uint) without manually converting values to byte arrays.

Note: Configure the PLC IP and credentials for your device. The sample writes values to the connected PLC; use a safe test project.

10.6. STRUCT Access

Please reference the sample code Example5.cs. This example demonstrates synchronous read/write of a CODESYS STRUCT Symbol (Application.PLC_PRG.sym_struct).

The example uses ADC_DEMO_STRUCT with BYTE, WORD and DWORD fields, writes the complete structure and reads it back. The PLC and C# structure layouts must match (8 bytes, including padding).



Note: The PLC Symbol must be published and writable. Struct layout must be verified on the target PLC; a mismatch can cause incorrect field values even when the supplied buffer length is 8 bytes.

Additional Windows C# examples demonstrate synchronous Symbol and Memory access using byte arrays (Example4.cs), the generic APIs (Example4_2.cs), and synchronous STRUCT access (Example5.cs). Open the corresponding projects in Example.sln, configure the PLC IP address and credentials, and use a suitable CODESYS test project.

Note: Example4, Example4_2 and Example5 write values to the connected PLC. Use a safe test application and verify Symbol access rights before running them.

Example4.cs: Synchronous Symbol and Memory read/write; includes DataConnect.GetSymbolList() and DataConnect.SetMemConnectPort(). Symbol List memory is automatically released by the wrapper.

Example4_2.cs: Synchronous Symbol and Memory read/write using generic APIs and DataConnect.GetSymbolList().

Example5.cs: Synchronous read/write of a CODESYS STRUCT Symbol using DataConnect.ReadSymbolValueSync<T>() and DataConnect.WriteSymbolValueSync<T>().

11. ADC library error code

Error Code		
Code	Name	Description
0	ADC_SUCCESS	Action successful.
1001	ADC_HANDLE_INVALID	The connection handle is invalid. Typically the wrong connection name.
1002	ADC_PARAMETER_INVALID	At least one parameter is invalid.
1003	ADC_THREAD_RUNNING	The ADC thread is running.
1004	ADC_THREAD_STOP	The ADC thread is stop.
1005	ADC_MEMALLOC_FAILED	Failed to allocate memory.
1006	ADC_OBJECT_DUPLICATED	The specified object already exists.
1007	ADC_OBJECT_NOT_EXIST	The specified object does not exist.
1008	ADC_SETPRIO_FAILED	Failed to set the ADC thread priority.
1009	ADC_CREATE_THREAD_FAILED	Failed to create the ADC connection.
1010	ADC_EXCEED_SECTOR_SIZE	The total size of the memory item exceeds the sector size.
1011	ADC_BUFFER_TOO_SMALL	The provided buffer is smaller than the Symbol data size.
1012	ADC_SIZE_MISMATCH	The provided data size does not match the Symbol data size.
1013	ADC_INCOMPLETE_DATA	The communication response does not contain complete data.
1014	ADC_WRONG_CONTEXT	The API was called from an unsupported thread context.
2001	ADC_PLC_CONNECTED	PLC is connected, symbols are loaded and all of them are verified. Now you can work correctly with the ADC symbol object.
2002	ADC_PLC_NOT_CONNECTED	PLC is not connected.
2003	ADC_PLC_NO_SYMBOLS	No symbols are available or symbol

		mismatch because of changed project.
2004	ADC_PLC_SYMBOLS_LOADED	PLC is connected and symbols are loaded.
2005	ADC_PLC_DISCONNECT	Connection is just getting terminated.
2006	ADC_PLC_NO_CONFIGURATION	Configuration of the ADC is invalid.
2007	ADC_PLC_NOT_CONNECTED_SYMBOLS_LOADED	PLC is currently not available, but the symbol information could be loaded offline. The ADC tries further to connect to the PLC.
2008	ADC_PLC_SYMBOL_NOT_MATCH	The symbol sector contains invalid elements or the symbol sector does not match to the actual symbol list.
2009	ADC_PLC_SYMBOL_SIZE_NOT_MATCH	The variable size is less than the actual symbol size
2010	ADC_PLC_SYMBOL_VALUE_NOT_READY	Symbol sector was defined and ADC thread is running, but the first read of the symbol sector has not yet been finished. Try again later to read the variables.
2011	ADC_PLC_NOT_VALID_DEVICE	Target device is not Advantech RTE.
2012	ADC_PLC_SYMBOL_LOAD_FAILED	Failed to load Symbols from the PLC.
2099	ADC_PLC_INTERNAL_ERROR	PLCHandler internal error.
3001	ADC_MEM_CONNECTED	Connect to the ADC server successes.
3002	ADC_MEM_WSASTART_FAILED	Failed to initial ADC resource.
3003	ADC_MEM_WSACLEAN_FAILED	Failed to terminate ADC resource
3004	ADC_MEM_CREATESOCK_FAILED	Failed to create socket to connect to ADC server.
3005	ADC_MEM_WAIT_TIMEOUT	Timeout exceeded.
3006	ADC_MEM_CONNECT_FAILED	Failed to connect to the ADC server.
3007	ADC_MEM_TCPSEND_FAILED	Failed to send packet to ADC server.
3008	ADC_MEM_TCPRECV_FAILED	Failed to receive packet from ADC server
3009	ADC_MEM_SOCKET_CLOSED	The connection does not connect.
3010	ADC_MEM_THREAD_BUSY	The previous write memory operation was not completed
3901	ADC_MEM_MODEXCEPT_01	Illegal function

3902	ADC_MEM_MODEXCEPT_02	Illegal data address
3903	ADC_MEM_MODEXCEPT_03	Illegal data value
3904	ADC_MEM_MODEXCEPT_04	Slave device failure
3905	ADC_MEM_MODEXCEPT_05	Acknowledge
3906	ADC_MEM_MODEXCEPT_06	Slave device busy
3907	ADC_MEM_MODEXCEPT_07	Negative acknowledge
3908	ADC_MEM_MODEXCEPT_08	Memory parity error
3999	ADC_MEM_MODEXCEPT_99	Invalid packet

Troubleshooting

1. PLC channel information

When you have ruled out the network problem but still cannot connect to RTE through ADC library. You can check whether all communication channels are occupied.

Open the **CODESYS Project** and then choose the **PLC Shell** tab. Enter the **channelinfo** command to check whether there are still channels displayed as FREE. If none of them appear as FREE, please try to close unnecessary connections. Each ADC connection will occupy a channel.

